Codo Learn – Product Requirements Document (PRD)

Version: 0.1 (Draft) | **Date:** October 25, 2026 | **Status:** For review

> **Basis and assumptions.** The repository (github.com/osamasmsmsm666-png/codo-learn) is an early Next.js project with a default README, so the product vision here is inferred from the name and tech stack. Items marked **[Assumption]** need confirmation from the product owner.

1. Overview

1.1 Product Summary

Codo Learn is a modern, interactive web platform that teaches programming by doing. Learners follow short, structured lessons, write code directly in the browser, get instant feedback, and see their progress and streaks grow.

1.2 Vision

Make learning to code feel as engaging and approachable as a well-designed game, so that anyone can go from "I have never written code" to "I built something" without installing anything.

1.3 Problem Statement

Beginners abandon programming courses because of: (1) setup friction, (2) passive video or text content with little practice, (3) unclear next steps, and (4) lack of motivation and feedback. Many existing platforms are either too heavy and expensive or too shallow.

1.4 Goals and Non-Goals

Goals	Non-Goals (MVP)
Zero-setup, in-browser learning	Native mobile apps
Hands-on practice in every lesson	Live classes or video hosting
Clear progress and motivation loops	Marketplace for third-party instructors
Fast, accessible, beautiful UI	Job placement or recruiting features

Goals	Non-Goals (MVP)
Easy content authoring for admins	Full IDE / project hosting

2. Target Users

2.1 Personas

Layla, 19, university student [Assumption]. Studying a non-CS major, wants to learn Python and JavaScript for her own projects. Uses her phone and laptop. Needs short lessons and instant feedback; gives up if setup is hard.

Omar, 28, career switcher. Works full time, studies 30 minutes per evening. Wants a clear path and visible progress to stay motivated.

Sara, content admin. Creates and updates lessons. Needs a simple editor and the ability to publish without involving a developer.

2.2 Key Jobs To Be Done

- "When I have 20 free minutes, I want to learn one small concept and practice it, so I feel progress."
- "When my code is wrong, I want to understand why, so I can fix it myself."
- "When I return after a break, I want to resume exactly where I stopped."

3. Market Context and Differentiation

Competitors include freeCodeCamp, Codecademy, W3Schools, Scrimba and Khan Academy. Codo Learn can differentiate through:

1. **Speed and polish:** a fast, animated, mobile-first UI.
2. **Bite-sized practice loops:** every lesson ends with a runnable exercise.
3. **Motivation design:** streaks, XP and badges tuned to short daily sessions.
4. **Localization [Assumption]:** strong Arabic/English bilingual support with RTL, an underserved niche.

4. Product Scope and Release Plan

4.1 MVP (Release 1.0)

Epic	Description	Priority
Accounts	Sign up, sign in, profile, password reset	Must
Catalog	Browse and enroll in courses	Must
Lesson viewer	Rich lessons with code samples, navigation, resume	Must
Exercises	In-browser editor, run code, test-based feedback	Must
Dashboard	Progress, continue-learning card, recent activity	Must
Admin CMS	Create and publish courses, lessons, exercises	Must
Gamification (light)	XP and daily streak	Should
Dark mode, responsive UI	Core polish	Should

4.2 Release 1.1 – Engagement

Badges, certificates, bookmarks and notes, progressive hints, quizzes, email reminders.

4.3 Release 2.0 – Growth

More languages and tracks, projects and challenges, leaderboards, community Q&A, AI tutor for hints and code explanations **[Assumption]**, payments and premium plans.

4.4 Indicative Timeline

Phase	Duration	Outcome
Discovery and design	2 weeks	Confirmed scope, wireframes, design system
Foundation	2 weeks	Auth, database, CI, content model
Learning core	4 weeks	Catalog, lesson viewer, exercise runner
Admin and progress	3 weeks	CMS, dashboard, XP and streaks
Hardening and beta	3 weeks	Accessibility, performance, security, closed beta
Launch	1 week	Public release

Total: roughly 15 weeks for a small team **[Assumption: 2 developers, 1 designer part time]**.

5. User Stories and Acceptance Criteria

ID	User story	Acceptance criteria
US-01	As a visitor, I want to browse courses without an account so I can decide if the platform fits me.	Catalog and course previews are public; sign-up prompt appears only when enrolling.
US-02	As a learner, I want to register and sign in so my progress is saved.	Email sign-up works; invalid input shows clear errors; session persists across visits.
US-03	As a learner, I want to read a lesson and run example code so I understand the concept.	Lesson shows formatted text and highlighted code; example code can be run and edited.
US-04	As a learner, I want to solve an exercise and get instant feedback so I know if I am right.	Results appear in under 3 s; each test shows pass or fail with a helpful message.
US-05	As a learner, I want to resume where I left off so I do not waste time.	Dashboard "Continue" button opens the last incomplete lesson.
US-06	As a learner, I want XP and streaks so I stay motivated.	XP awarded on completion; streak increments once per day and resets after a missed day.
US-07	As a learner on my phone, I want a usable editor so I can practice anywhere.	Layout adapts at 360 px width; editor and output are reachable without horizontal scroll.
US-08	As an admin, I want to publish a lesson without a code deploy.	New lesson appears in the catalog within a minute of publishing.
US-09	As an admin, I want to define test cases for an exercise so grading is automatic.	Tests can be added, edited and previewed before publishing.
US-10	As a learner, I want to delete my account so I control my data.	Account and personal data are removed after confirmation.

6. Key User Flows

1. **Onboarding:** Landing → choose a goal or language → sample lesson (no sign-up) → prompt to save progress → register → dashboard.

2. **Daily learning loop:** Dashboard → Continue → read lesson → attempt exercise → feedback and hints → complete → XP and streak update → next lesson.
3. **Content publishing:** Admin sign-in → create lesson in Markdown → add exercise and tests → preview → publish.

7. Functional Requirements Summary

Detailed, numbered requirements are in the companion *Codo Learn SRS*. In summary: authentication and roles, course catalog and enrollment, lesson viewer, sandboxed code execution with test-based grading, progress tracking with XP and streaks, admin content management, responsive and accessible UI.

8. Non-Functional Requirements Summary

Area	Target
Performance	LCP under 2.5 s on mid-range mobile; exercise feedback under 3 s
Accessibility	WCAG 2.2 AA; keyboard operable; reduced-motion support
Security	Sandboxed code execution, hashed passwords, OWASP protections
Availability	99.5% monthly uptime
Privacy	Data minimization, consent, export and deletion
Scalability	1,000 concurrent learners at launch

9. Technical Approach (High Level)

Concern	Current / proposed
Frontend	Next.js 16 App Router, React 19, TypeScript, Tailwind CSS 4, Motion, lucide-react (already in repo)
Content	MDX files in the repo for MVP, or database-backed CMS if non-developers must publish (see Open Questions)

Concern	Current / proposed
Backend	Next.js route handlers or server actions
Data	PostgreSQL with an ORM (e.g., Prisma or Drizzle) **[Proposed]**
Auth	Managed auth library or provider with email and OAuth **[Proposed]**
Code execution	In-browser Web Worker for JavaScript; sandbox API for other languages **[Proposed]**
Hosting	Vercel or equivalent Node host
Quality	ESLint, type-check, unit and end-to-end tests, CI pipeline

> Engineering note: the repository's AGENTS.md states this Next.js version has breaking changes. The team must consult the bundled docs in `node_modules/next/dist/docs/` before implementing features.

10. Success Metrics

Metric	Target (first 3 months after launch)
Activation: new users completing first lesson	60%
Day-7 retention	25%
Weekly active learners	1,000
Exercise first-attempt pass rate	40–60% (indicates balanced difficulty)
Course completion rate	15%
Median time to first exercise	Under 3 minutes
Lighthouse performance and accessibility scores	90+

All targets are placeholders to be calibrated once baseline data exists.

11. Risks and Mitigations

Risk	Impact	Mitigation
Unsafe code execution	High	Isolated sandboxes, resource limits, no server-side eval
Content creation is slow	High	MDX templates, start with one strong track, reuse exercise formats
Low retention	Medium	Streaks, reminders, short lessons, early user testing
Framework breaking changes	Medium	Pin versions, follow bundled docs, add CI checks
Scope creep	Medium	Strict MVP scope, quarterly roadmap review
Competition from free resources	Medium	Focus on UX quality and localization

12. Dependencies

Database and hosting accounts, email provider, sandbox provider (if used), design assets, and initial learning content with reviewers.

13. Open Questions

1. Who exactly is the audience (age range, language, experience)?
2. Which language or track comes first (HTML/CSS, JavaScript, Python)?
3. What is the business model (free, freemium, subscription, sponsored)?
4. Is Arabic/RTL required for launch?
5. Will content be in the repository (MDX) or in an admin-managed database?
6. Is an AI tutor feature desired, and with what budget?

14. Approvals

Role	Name	Date	Signature
Product owner			
Engineering lead			

Role	Name	Date	Signature
Design lead			